

(JavaScript + Node.js) — Assignment

Q1. JavaScript Coding Practice

Below are five short JavaScript programs, each with the problem statement, source code, sample output, and a simple explanation.

1. Check whether a number is even or odd

Problem Statement: Write a program that takes a number and checks whether it is even or odd.

```
function checkEvenOdd(num) {  
  if (num % 2 === 0) {  
    console.log(num + " is Even");  
  } else {  
    console.log(num + " is Odd");  
  }  
}  
  
checkEvenOdd(7);  
checkEvenOdd(10);
```

Output:

```
7 is Odd  
10 is Even
```

Explanation: The % (modulus) operator gives the remainder after dividing by 2. If the remainder is 0, the number is even; otherwise, it is odd.

2. Find the largest of three numbers

Problem Statement: Write a program that takes three numbers and finds the largest among them.

```
function findLargest(a, b, c) {  
  let largest = a;  
  if (b > largest) {  
    largest = b;  
  }  
  if (c > largest) {  
    largest = c;  
  }  
  return largest;  
}  
  
console.log(findLargest(12, 45, 30));
```

Output:

45

Explanation: We assume the first number is the largest, then compare it with the other two, updating 'largest' whenever a bigger number is found.

3. Reverse a string

Problem Statement: Write a program that reverses a given string.

```
function reverseString(str) {  
  return str.split("").reverse().join("");  
}  
  
console.log(reverseString("Hello"));
```

Output:

olleH

Explanation: `split("")` breaks the string into an array of characters, `reverse()` flips the order of the array, and `join("")` joins the characters back into a single string.

4. Count vowels in a string

Problem Statement: Write a program that counts how many vowels (a, e, i, o, u) are present in a string.

```
function countVowels(str) {  
  const vowels = "aeiouAEIOU";  
  let count = 0;  
  for (let ch of str) {  
    if (vowels.includes(ch)) count++;  
  }  
  return count;  
}  
  
console.log(countVowels("JavaScript Programming"));
```

Output:

7

Explanation: The program loops through each character of the string and checks if it exists inside the 'vowels' string. Every match increases the counter.

5. Remove duplicate values from an array

Problem Statement: Write a program that removes duplicate values from an array.

```
let arr = [1, 2, 2, 3, 4, 4, 5];
let result = [];

for (let i = 0; i < arr.length; i++) {
  if (!result.includes(arr[i])) {
    result.push(arr[i]);
  }
}

console.log(result);
```

Output:

[1, 2, 3, 4, 5]

Explanation: We start with the original array 'arr' and an empty array 'result'. We loop through each element of 'arr' one by one, and check if it is not already present in 'result' using includes(). If it is new, we push it into 'result'; if it is a repeat, we skip it. After the loop finishes, 'result' holds only the unique values.

Q2. Introduction to Node.js

What is Node.js?

Node.js is a free, open-source, cross-platform runtime environment that allows JavaScript to run outside of a web browser, directly on a computer or server. It is built on Google Chrome's V8 JavaScript engine and is mainly used to build fast, scalable server-side and network applications, such as APIs, backend services, and automation scripts.

How does Node.js work?

Node.js works using a single-threaded, event-driven, non-blocking I/O model:

- It runs on a single main thread, but instead of waiting for slow tasks (like reading a file or calling a database) to finish, it hands them off and moves on to the next task.
- Node.js uses an Event Loop that constantly checks for completed tasks and runs their callback functions when ready.
- This non-blocking approach allows Node.js to handle many requests at the same time without creating a new thread for each one, making it lightweight and efficient.

Advantages of Node.js for QA Automation

- Same language as many test frameworks: JavaScript is used in popular automation tools (e.g., Playwright, Cypress, WebdriverIO), so testers can write both frontend and backend/test scripts in one language.
- Fast execution: The non-blocking, event-driven architecture makes running large test suites quicker, especially for API testing.
- Huge package ecosystem (npm): Thousands of ready-made libraries for testing, reporting, mocking, and API requests are available, saving development time.
- Easy integration with CI/CD: Node.js-based test scripts integrate smoothly with tools like Jenkins, GitHub Actions, and GitLab CI.
- Good for API and performance testing: Its asynchronous nature makes it well-suited for sending many concurrent requests during load or API testing.

Difference between Browser JavaScript and Node.js

Aspect	Browser JavaScript	Node.js
Environment	Runs inside a web browser (Chrome, Firefox, etc.)	Runs outside the browser, directly on a machine/server
Engine	Uses the browser's built-in JS engine (e.g., V8 in Chrome)	Uses the standalone V8 engine bundled with Node.js
DOM Access	Can access and manipulate the DOM (HTML/CSS elements)	No DOM access; there is no browser window or document
File System Access	Cannot directly read/write files on the computer (for security)	Can read/write files using built-in modules like 'fs'
Modules	Uses ES6 import/export or <script> tags	Uses require()/module.exports (CommonJS) or ES6 modules
Typical Use	Making web pages interactive (forms, animations, events)	Building servers, APIs, automation scripts, and backend logic